Architecture Document — Listicle Generator

Elevate Group / WideStep AI Automation Engineer Assessment

Built by Valentina P.

1. High-Level System Design and Data Flow

The system is built as a lightweight monolithic web app to keep local execution simple,one command and it runs.

Frontend: Two pages built with HTML, CSS and Vanilla JS. The Create page takes the inputs. The Dashboard shows all generated listicles with their status and a preview button.

Backend: Python and Flask. It acts as the orchestrator receives the inputs, manages background jobs, and serves the generated files.

Database: Local SQLite. Stores job status (Pending, Completed, Error) and the file paths of generated pages.

Pipeline:

- **Ingest:** The user submits the research JSON, the product URL, and the reference URL. The system registers the job in SQLite.
- **Scraping:** BeautifulSoup visits the product URL and dynamically extracts image and video URLs.
- **AI Orchestration (Groq API):** The research JSON is sent to an LLM via Groq. Groq was chosen for its low latency and large context window (128k tokens), which handles the full research JSON without truncation. The orchestration layer is interchangeable with any OpenAI-compatible provider. The model is prompted strictly to return structured JSON — headlines, copy, benefit sections, testimonials, FAQ, and CTA.
- **Assembly (Jinja2):** A Jinja2 template takes the HTML skeleton based on the reference site structure and injects the AI-generated copy and scraped media.
- **Export:** The final .html file is saved to disk and served via Flask. The dashboard updates automatically.

2. Likely Failure Modes and Handling

Scraping fails (DOM changes): If the product page structure changes and images can't be extracted, the job doesn't crash. The scraper has try-except blocks and continues with whatever it found, injecting placeholder images if needed.

LLM hallucinations or bad format: The model could return plain text instead of the expected JSON structure. The prompt is very restrictive and demands valid JSON only. If parsing fails, the system retries automatically. If it fails twice, the job is marked as error and the raw output is saved for debugging.

UI timeouts: Scraping and AI generation can take a few seconds. The frontend polls for status every 3 seconds and shows a loading spinner while disabling the submit button to prevent duplicate submissions.

3. Assumptions made for this MVP

1. The reference URL is used to manually extract the structural skeleton (layout, containers, hierarchy) upfront and build a Jinja2 template from it, rather than having the AI generate CSS from scratch on every run. This guarantees a conversion-ready design with consistent quality.

2. Complex interactive elements from the reference (like countdown timers) are replaced with static versions to keep the focus on structure and copy.

3. Generated HTML files are served via Flask using send_from_directory from a public output folder, previewable from the dashboard in a new tab.

.